



Budapesti Gazdasági SZC

Logisztikai és Kereskedelmi Technikum és Szakképző Iskola

Szoftverfejlesztő és -tesztelő technikus

5 0613 12 03

CyberNest

Készítette: Macza Ádám
Király Gellért
Kiss Richárd Bendegúz

Budapest, 2026

Tartalom

1	Bevezetés	1
1.1	Feladat Leírás	1
1.2	Célcsoport	1
2	Követelményelemzés	3
2.1	Funkcionális követelmények	3
2.2	Nem funkcionális követelmények	3
2.3	Felhasználói Szerepkörök	4
2.4	Use-case áttekintés	4
2.5	Működési Logika.....	5
2.6	Követelménymátrix	5
2.7	Felhasználói Felület Vázlatai (Wireframe)	7
2.7.1	1. Főoldal	7
2.7.2	2. Terméklista oldal	7
2.7.3	3. Foglalási folyamat.....	8
2.7.4	4. Ügyfélportál	8
3	Adatbázis-terv	10
3.1	Entitások és kapcsolatok	10
3.2	Adattípusok, kulcsok, validációk	11
3.3	ER diagram	12
4	Backend Fejlesztés (REST API).....	13
4.1	Technológiai választás.....	13
4.2	Szerveroldali architektúra	13
4.3	Hitelesítés és jogosultságkezelés	13
4.4	REST API végpontok	14
4.5	Végpontok listája.....	14
4.5.1	Auth	14
4.5.2	Eszközök.....	14
4.5.3	Időpontok	15
4.5.4	Foglalások	16
4.5.5	Felhasználók	16
4.5.6	Üzemeltetők	17
4.5.7	Értékelések	17
4.5.8	Logok	18

4.5.9	Speciális végpontok	19
4.6	A Felhasznált Ismeretek	19
4.7	A Felhasznált Szoftverek	20
4.8	Programozási nyelvek és technológiák	20
4.9	Könyvtárak, keretrendszerek	20
4.10	Fejlesztői környezet és eszközök.....	21
5	Frontend Fejlesztés	23
5.1	Webes kliens	23
5.1.1	Fő útvonalak	23
5.1.2	API integráció.....	24
5.2	Mobil kliens -nincs-	24
5.3	Asztali kliens	24
5.4	Fontos frontend funkciók.....	24
5.5	Fontos megjegyzések a jelenlegi implementációról	25
6	Asztali Alkalmazás Fejlesztés	26
6.1	Áttekintés	26
6.2	Technológia	26
6.3	Fő funkciók	26
6.4	Felépítés	27
6.5	Működés.....	28
6.6	Fontos technikai megfigyelések	29
7	Projektmenedzsment.....	30
7.1	Projekttervezés	30
7.2	Célkitűzések és határidők.....	30
7.3	Csapatmunka és Verziókövetés.....	31
7.3.1	Feladatmegosztás	31
7.3.2	Verziókövetés	31
7.4	Kockázatelemzés	31
8	Fejlesztői Dokumentáció	33
8.1	Részletes feladatspecifikáció.....	33
8.2	Entitások leírása	33
8.2.1	Eszköz entitás	33
8.2.2	Felhasználó entitás	33
8.2.3	Foglalás entitás	33

8.2.4	Kapcsolat az entitások között	33
8.3	Használati esetek (use-case)	34
8.3.1	Vendég.....	34
8.3.2	Regisztrált felhasználó	34
8.3.3	Admin	34
8.4	Nézetek	34
8.5	API végpontok meghatározása.....	34
8.6	Projektstruktúra	34
8.7	Környezeti változók	38
8.8	Részletes feladat-specifikáció, algoritmusok.....	39
8.8.1	1. Regisztráció.....	39
8.8.2	2. Bejelentkezés.....	39
8.8.3	3. Foglalás létrehozása	39
8.8.4	4. Mentorértékelés mentése.....	40
8.9	Az alkalmazott fejlesztői eszközök	40
8.10	Fejlesztői megjegyzések	40
9	Tesztelési Dokumentáció.....	42
9.1	Általános tesztelési stratégia	42
9.2	Tesztelési táblázat	42
9.3	Tesztesetek.....	43
9.3.1	1. Normál használat - érvényes regisztráció.....	43
9.3.2	2. Negatív teszt - hibás e-mail cím.....	43
9.3.3	3. Bolondbiztos teszt - túl nagy profilkép.....	43
9.4	Automatizált tesztelés.....	44
9.5	Technikai ellenőrzések (2026. április 7.)	44
9.6	Talált hibák	44
9.7	Tesztelési környezet.....	45
10	Felhasználói Dokumentáció.....	46
10.1	A program általános specifikációja	46
10.2	Rendszerkövetelmények	46
10.2.1	Hardverkövetelmények	46
10.2.2	Szoftverkövetelmények	46
10.3	A teljes program telepítése.....	47
10.3.1	A weboldal és backend - 1. Módszer: batch fájlal közvetlen repóból (Ajánlott*).47	

10.3.2	A weboldal és backend - 2. Módszer: kézi indítás (Nem ajánlott)	47
10.3.3	Az asztali alkalmazás telepítése közvetlen a repóból letöltve (Nem ajánlott)	47
10.3.4	A Teljes projekt Egyszerű szakszerű telepítése (Feltétlenül ajánlott!!).....	49
10.4	Fontos telepítési megjegyzések	50
10.5	A program használatának részletes leírása	50
10.5.1	1. Főoldal használata	50
10.5.2	2. Termékek böngészése	50
10.5.3	3. Regisztráció és bejelentkezés	51
10.5.4	4. Foglálási folyamat.....	51
10.5.5	5. Dashboard használata	51
10.5.6	6. Profil használata	51
10.5.7	7. Desktop admin kliens használata	51
10.6	FAQ.....	52
11	Összefoglalás	53
11.1	Önértékelés	53
11.2	Továbbfejlesztési Lehetőségek.....	53
12	Felhasznált Irodalom.....	55
12.1	Nyomtatott könyv	55
12.2	Elektronikus dokumentumok és projektforrások	55
12.3	Internetes források (weboldal, dokumentációk)	55
13	Ábrajegyzék.....	57

CyberNest Vizsgaremek Teljes Dokumentáció

Csapattagok és szerepkörök

Név	Feladatkör
Király Gellért	Frontend, Backend, Database
Macza Ádám	Frontend, Backend, Database
Kiss Richárd Bendegúz	Desktop

Megjegyzés: a repository jelenlegi állapotában a webes frontend és backend mellett külön asztali admin kliens is megtalálható CyberNest_Admin néven. A teljes dokumentáció ezért már a webes és a desktop modulokat együtt tárgyalja.

1 Bevezetés

A CyberNest egy többmodulos vizsgaremek-projekt, amelynek célja egy modern, reszponzív, adatbázis-alapú webalkalmazás létrehozása. A rendszer lehetővé teszi, hogy a felhasználók szerverkonfigurációkat böngésszenek, részletes műszaki adatokat tekintsenek meg, mentor segítségével időpontot foglaljanak, majd a saját foglalásaikat kezeljék és a választott mentort értékeljék.

A megoldás kliens-szerver architektúrára épül. A frontend React + Vite környezetben készült, a backend Node.js + Express alapú REST API, az adattárolást pedig MySQL biztosítja. A projekt támogatja a hagyományos regisztrációs/bejelentkezési folyamatot, valamint a Google- és GitHub-alapú OAuth belépést is. A hitelesítés JWT tokenekkel történik.

A dokumentáció a repository 2026. április 7-i állapotára épül, tehát nem egy elméleti tervet, hanem a jelenleg megtalálható kódbázis tényleges működését írja le.

1.1 Feladat Leírás

A projekt fő célja egy olyan online platform megvalósítása, amely egyszerűen kezelhető felületen teszi lehetővé infrastruktúra-jellegű szervercsomagok kiválasztását és mentorált foglalását. A rendszer az alábbi problémákra ad megoldást:

- a felhasználó ne csak listát lásson az elérhető konfigurációkról, hanem szűrhető, rendezhető formában hasonlíthassa össze őket;
- a kiválasztott konfigurációhoz rendelhető legyen mentori segítség;
- a foglalási folyamat több lépésben, validációkkal védve történjen;
- a felhasználó saját profilját, foglalásait és értékeléseit egy helyen tudja kezelni;
- az adminisztrátorok külön API-végpontokon keresztül tudják kezelni az adatbázisban tárolt erőforrásokat.

1.2 Célcsoport

- érdeklődő látogatók, akik szervercsomagokat böngésszenek;
- regisztrált ügyfelek, akik foglalást adnak le;
- adminisztrátorok, akik az adatokat kezelik;

Bevezetés

- a szolgáltatáshoz kapcsolódó mentorok, mint üzleti szereplők.

2 Követelményelemzés

2.1 Funkcionális követelmények

- szerverkonfigurációk listázása adatbázisból;
- szerverek részletes nézete;
- regisztráció és bejelentkezés;
- OAuth bejelentkezés Google és GitHub szolgáltatóval;
- védett foglalási folyamat;
- elérhető időpontok lekérdezése és lefoglalása;
- felhasználói profil szerkesztése;
- foglalások listázása és törlése;
- mentorértékelés mentése és átlagpontszám megjelenítése;
- adminisztrációs API-végpontok biztosítása.

2.2 Nem funkcionális követelmények

- reszponzív, modern felhasználói felület;
- REST API alapú kommunikáció JSON formátumban;
- jogosultságkezelés JWT tokennel;
- MySQL adatbázis és relációs modell;
- tiszta, moduláris kódszerkezet;
- kliens és szerver szétválasztása egy közös repón belül;
- alapvető bemeneti validációk és hibakezelés.

2.3 Felhasználói Szerepkörök

Szerepkör	Leírás	Jogosultságok
Vendég	Nem bejelentkezett látogató	Főoldal, terméklista, termék részlet, kapcsolat oldal, regisztráció, login
Regisztrált felhasználó	Hitelesített ügyfél	Foglalás indítása, saját foglalások megtekintése/törlése, profil módosítása, mentor értékelése
Admin	Emelt jogosultságú felhasználó	Felhasználók, eszközök, időpontok, üzemeltetők, értékelések, logok admin API-kezelése
Mentor	Üzleti szereplő, jelenleg statikus adatkészletből	A jelenlegi verzióban nem külön belépési role, de a foglalás részeként kiválasztható

2.4 Use-case áttekintés

flowchart LR

V((Vendég))

F((Felhasználó))

A((Admin))

V --> U1["Termékek böngészése"]

V --> U2["Regisztráció / bejelentkezés"]

V --> U3["Kapcsolat oldal megnyitása"]

F --> U4["Foglalás indítása"]

F --> U5["Mentor választása"]

F --> U6["Időpont foglalása"]

F --> U7["Foglalások kezelése"]

F --> U8["Profil módosítása"]

F --> U9["Mentor értékelése"]

```
A --> U10["Felhasználók kezelése"]
A --> U11["Eszközök kezelése"]
A --> U12["Időpontok kezelése"]
A --> U13["Üzemeltetők kezelése"]
A --> U14["Értékelések és logok lekérdezése"]
```

2.5 Működési Logika

1. A látogató megnyitja a főoldalt, ahol megjelenik a marketing jellegű nyitóképernyő, az előnyök blokk, a visszajelzések és a beágyazott terméklista.
2. A termékoldalon a rendszer a backend `/api/eszkoz` végpontjáról betölti az adatbázisban tárolt szervereket.
3. A felhasználó szűréssel, kereséssel és rendezéssel kiválaszthatja a számára megfelelő szervert.
4. A részletes nézetben megtekintheti a konfiguráció jellemzőit és elindíthatja a foglalást.
5. Foglalás csak hitelesített felhasználóként indítható, ezért a rendszer védett útvonalat használ.
6. A foglalási folyamatban a felhasználó először mentort választ, majd a szerverhez tartozó elérhető időpontok közül foglal.
7. A felhasználó megadja kapcsolattartási, számlázási és egyéb foglalási adatait.
8. A rendszer ellenőrzi a bemenetek érvényességét, majd a kiválasztott időpontot lefoglalja és létrehozza a foglalást.
9. A felhasználó a saját vezérlőpultján megtekintheti és törölheti foglalásait.
10. Mentoros foglalás esetén a felhasználó értékelést küldhet, amelyet a rendszer egy külön `mentor_ertekeles` táblában tárol.

2.6 Követelménymátrix

Azo-nosító	Követelmény	Megvalósítás	Állapot
KF-01	Adatbázisból származó szerverlista	<code>GET /api/eszkoz</code> , <code>Products.jsx</code>	Teljesült

Követelményelemzés

Azo- nosító	Követelmény	Megvalósítás	Állapot
KF-02	Részletes termékoldal	GET /api/eszkoz/:id, ServerDetails.jsx	Teljesült
KF-03	Regisztráció és bejelentkezés	authController, Login.jsx, Register.jsx	Teljesült
KF-04	JWT-alapú jogosultságkezelés	middleware/auth.js, axios interceptor	Teljesült
KF-05	OAuth belépés	Google + GitHub callback útvonalak	Teljesült
KF-06	Többlépéses foglalási folyamat	BookingFlow.jsx, foglalasController.js	Teljesült
KF-07	Időpontkezelés	idopont tábla és API	Teljesült
KF-08	Saját foglalások kezelése	Dashboard.jsx, GET /api/foglalas/my, DELETE /api/foglalas/:id	Teljesült
KF-09	Mentorértékelés	mentor_ertekeles tábla és dashboard űrlap	Teljesült
KF-10	Profilkezelés avattarral	PUT /api/auth/me, Profile.jsx	Teljesült
KF-11	Kapcsolatfelvételi űrlap backend integrációval	Frontend űrlap jelen van, de csak statikus alert	Részben teljesült
KF-12	Admin CRUD funkciók	felhasználó, eszköz, időpont, üzemeltető, értékelés, log API-k	Teljesült

Azo- nosító	Követelmény	Megvalósítás	Állapot
KF-13	Automatizált tesztek	A repó nem tartalmaz valódi unit/integrációs teszteket	Nem teljesült
KF-14	Desktop admin kliens	WPF alapú admin alkalmazás bejelentkezéssel és CRUD funkciókkal	Részben teljesült

2.7 Felhasználói Felület Vázlatai (Wireframe)

2.7.1 1. Főoldal



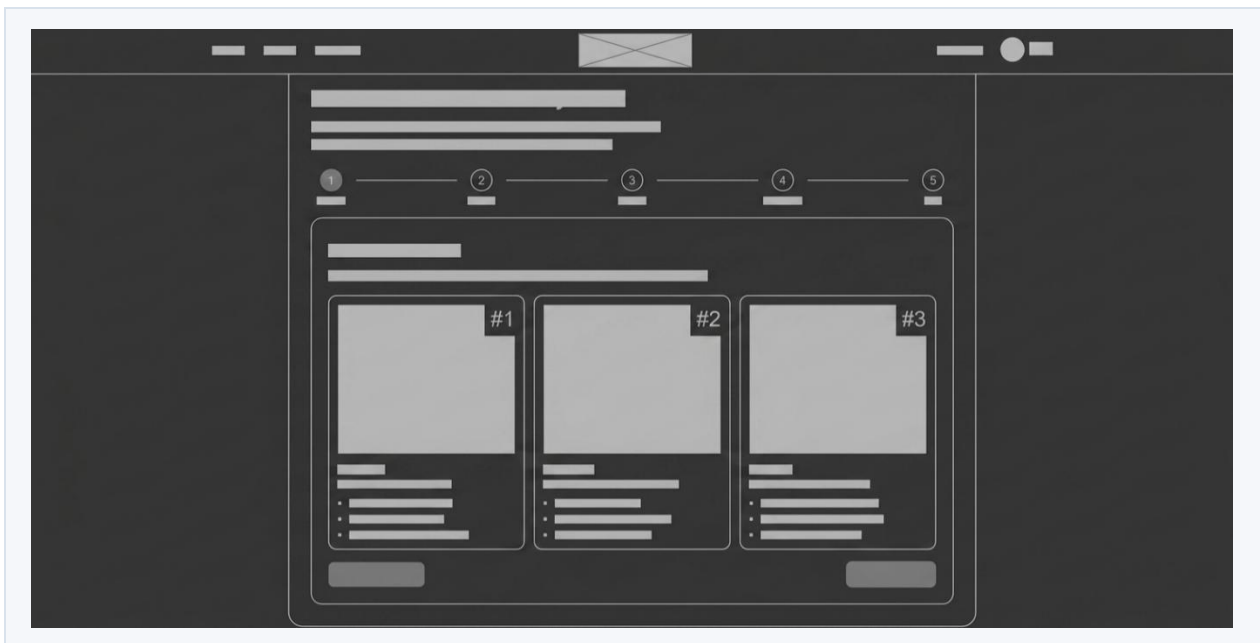
ábra 1 Főoldal egyszerű drótváza (WireFrame)

2.7.2 2. Terméklista oldal



ábra 2 Terméklista oldal egyszerű drótváza (WireFrame)

2.7.3 3. Foglalási folyamat



ábra 3 Foglalási folyamat oldal egyszerű drótváza (WireFrame)

2.7.4 4. Ügyfélportál



ábra 4 Ügyfélportál oldal egyszerű drótváza (WireFrame)

3 Adatbázisterv

A rendszer relációs adatbázist használ MySQL alapon. A backend indulásakor a `setup.sql` szkript automatikusan létrehozza a szükséges adatbázist és táblákat, illetve betölti a demo adatokat.

3.1 Entitások és kapcsolatok

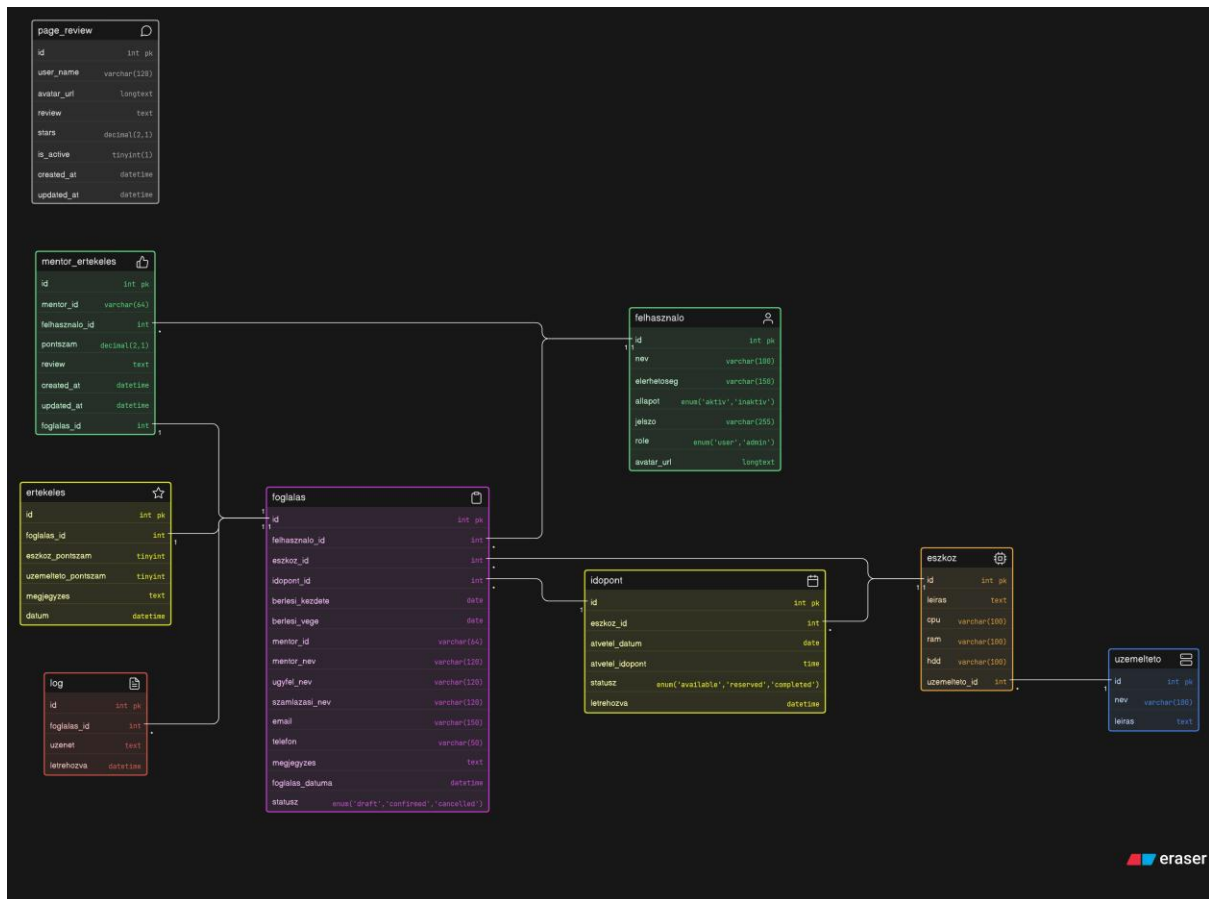
Tábla	Cél	Fő mezők	Kapcsolatok
uzemelteto	Szolgáltatók, üzemeltetők tárolása	id, nev, leiras	1:N kapcsolat az eszkoz táblával
eszkoz	Szerverkonfigurációk	id, leiras, cpu, ram, hdd, uzemelteto_id	N:1 uzemelteto, 1:N idopont, 1:N foglalas
felhasznalo	Rendszerfelhasználók	id, nev, elerhetoseg, allapot, jelszo, role, avatar_url	1:N foglalas, 1:N mentor_ertekeles
idopont	Foglalható időpontok	id, eszkoz_id, atvetel_datum, atvetel_idopont, statusz	N:1 eszkoz, 1:0..1 foglalas
foglalas	Létrejött foglalások	id, felhasznalo_id, eszkoz_id, idopont_id, mentor_id, mentor_nev, ügyfél-adatok	N:1 felhasznalo, N:1 eszkoz, N:1 idopont, 1:1 ertekeles, 1:0..1 mentor_ertekeles, 1:N log
ertekeles	Foglalás utáni eszköz/üzemeltető értékelés	id, foglalas_id, eszkoz_pontszam, uzemelteto_pontszam, megjegyzes	1:1 foglalas

Tábla	Cél	Fő mezők	Kapcsolatok
men- tor_erte- keles	Mentor értékelés	id, foglalas_id, mentor_id, felhasznalo_id, pontszam, review	N:1 felhasznalo, 1:1 fogla- las
fake_re- view	Nyilvános kez- dőoldali vélemé- nyek	id, user_name, avatar_url, review, stars	Nincs közvetlen külső kulcs
log	Foglaláshoz tar- tozó esemény- napló	id, foglalas_id, uzenet, let- rehozva	N:1 foglalas

3.2 Adattípusok, kulcsok, validációk

- minden fő tábla elsődleges kulcsa id, automatikusan növekvő egész szám;
- az `eszkoz.uzemelteto_id`, `idopont.eszkoz_id`, `foglalas.felhasznalo_id`, `foglalas.eszkoz_id`, `foglalas.idopont_id`, `ertekeles.foglalas_id`, `mentor_ertekeles.felhasznalo_id`, `mentor_ertekeles.foglalas_id`, `log.foglalas_id` mezők idegen kulcsok;
- a `felhasznalo.role` mező `user` vagy `admin`;
- a `felhasznalo.allapot` mező `aktiv` vagy `inaktiv`;
- az `idopont.statusz` mező `available`, `reserved` vagy `completed`;
- a `foglalas.statusz` mező `draft`, `confirmed` vagy `cancelled`;
- a backend validációs rétege regexekkel ellenőrzi a neveket, e-mailt, telefonszámot és jelszót;
- a mentor review maximum 1200, az egyéb szöveges megjegyzések maximum 1000 karakteresek.

3.3 ER diagram



ábra 5 Az adatbázis ER Diagrammja.

4 Backend Fejlesztés (REST API)

4.1 Technológiai választás

- **Futtatókörnyezet:** Node.js
- **Keretrendszer:** Express 5
- **Adatbázis:** MySQL
- **Autentikáció:** JWT
- **Titkosítás:** bcrypt
- **Környezeti változók kezelése:** dotenv
- **Kommunikáció:** JSON alapú REST API

4.2 Szerveroldali architektúra

A backend réteges szerkezetben épül fel:

`routes/`: végpontdefiníciók;

`controllers/`: kérelmfeldolgozás, validáció, üzleti logika;

`models/`: adatbázis-elérés;

`middleware/`: tokenellenőrzés és admin jogosultság;

`config/`: adatbázis-kapcsolat és induláskori inicializálás;

`utils/`: közös validációs segédfüggvények.

4.3 Hitelesítés és jogosultságkezelés

- lokális regisztráció és belépés e-mail + jelszó párossal;
- JWT token 24 órás lejárattal;
- token küldése Authorization: Bearer <token> formában;
- admin jogosultság külön middleware-rel;
- Google és GitHub OAuth támogatás;
- a közösségi belépésnél a rendszer szükség esetén automatikusan létrehoz felhasználót.

4.4 REST API végpontok

4.5 Végpontok listája

4.5.1 Auth

Metódus	Végpont	Jogosultság	Funkció
POST	/api/auth/register	publikus	regisztráció
POST	/api/auth/login	publikus	bejelentkezés
GET	/api/auth/google/start	publikus	Google OAuth indítás
GET	/api/auth/google/callback	publikus	Google callback
GET	/api/auth/github/start	publikus	GitHub OAuth indítás
GET	/api/auth/github/callback	publikus	GitHub callback
GET	/api/auth/me	token	saját profil lekérdezése
PUT	/api/auth/me	token	saját profil módosítása

4.5.2 Eszközök

Metódus	Végpont	Jogosultság	Funkció
GET	/api/eszkoz	publikus	összes eszköz listázása

Backend Fejlesztés (REST API)

Metódus	Végpont	Jogosultság	Funkció
GET	/api/eszkoz/:id	publikus	egy eszköz részletei
POST	/api/eszkoz	admin	eszköz létrehozása
PUT	/api/eszkoz/:id	admin	eszköz módosítása
DELETE	/api/eszkoz/:id	admin	eszköz törlése

4.5.3 Időpontok

Metódus	Végpont	Jogosultság	Funkció
GET	/api/idopont/eszkoz/:eszkosz_id	publikus	elérhető időpontok lekérdezése eszközhöz
GET	/api/idopont	admin	összes időpont listázása
GET	/api/idopont/:id	publikus	időpont lekérdezése
POST	/api/idopont	admin	időpont létrehozása
PUT	/api/idopont/:id	admin	időpont módosítása
DELETE	/api/idopont/:id	admin	időpont törlése

4.5.4 Foglalások

Metódus	Végpont	Jogosultság	Funkció
POST	/api/foglalas	token	foglalás létrehozása
GET	/api/foglalas/my	token	saját foglalások lekérdezése
DELETE	/api/foglalas/:id	token	saját vagy admin által törölt foglalás
GET	/api/foglalas	admin	összes foglalás

4.5.5 Felhasználók

Metódus	Végpont	Jogosultság	Funkció
GET	/api/felhasznalo	admin	összes felhasználó
POST	/api/felhasznalo	admin	felhasználó létrehozása
GET	/api/felhasznalo/:id	token	felhasználó lekérdezése
PUT	/api/felhasznalo/:id	token	felhasználó módosítása
DELETE	/api/felhasznalo/:id	admin	felhasználó törlése

4.5.6 Üzemeltetők

Metódus	Végpont	Jogosultság	Funkció
GET	/api/uzemelteto	publikus	összes üzemeltető
GET	/api/uzemelteto/:id	publikus	egy üzemeltető
POST	/api/uzemelteto	admin	létrehozás
PUT	/api/uzemelteto/:id	admin	módosítás
DELETE	/api/uzemelteto/:id	admin	törlés

4.5.7 Értékelések

Metódus	Végpont	Jogosultság	Funkció
GET	/api/ertekeles	admin	összes értékelés
GET	/api/ertekeles/public/fake	publikus	kezdőoldali fake review-k
GET	/api/ertekeles/mentor/:mentor_id/atlag	token	mentor átlagpontszáma
GET	/api/ertekeles/mentor/foglalas/:foglalas_id	token	mentorértékelés foglalás szerint

Backend Fejlesztés (REST API)

Metódus	Végpont	Jogosultság	Funkció
POST	/api/ertekeles/mentor	token	mentorértékelés létrehozása vagy frissítése
GET	/api/ertekeles/foglalas/:foglalas_id	token	eszköz/üzemeltető értékelés foglalás szerint
GET	/api/ertekeles/:id	token	egy értékelés
POST	/api/ertekeles	token	értékelés létrehozása
PUT	/api/ertekeles/:id	token	értékelés módosítása
DELETE	/api/ertekeles/:id	token	értékelés törlése

4.5.8 Logok

Metódus	Végpont	Jogosultság	Funkció
GET	/api/log	admin	összes log
GET	/api/log/foglalas/:foglalas_id	token	logok foglalás szerint
GET	/api/log/:id	token	egy log
POST	/api/log	token	log létrehozása

Metódus	Végpont	Jogosultság	Funkció
PUT	/api/log/:id	token	log módosítása
DELETE	/api/log/:id	token	log törlése

4.5.9 Speciális végpontok

GET /api/health: egyszerű health check;

GET /api/debug/users: fejlesztői debug végpont a felhasználók listázásához;

GET /: HTML-alapú gyors adatbázis-összefoglaló nézet.

4.6 A Felhasznált Ismeretek

A projekt elkészítéséhez az alábbi tantárgyi és szakmai területek tudása került felhasználásra:

- HTML5 szemantikus felépítés;
- CSS3 és reszponzív webdesign;
- modern JavaScript (ES modulok, async/await, fetch/axios);
- React komponensalapú frontend-fejlesztés;
- React Router útvonalkezelés;
- állapotkezelés Context API-val;
- Node.js + Express backend fejlesztés;
- REST API tervezés;
- SQL és relációs adatmodellezés;
- autentikáció és jelszóhash-elés;
- verziókövetés Git használatával;
- hibakezelés, validáció, alapvető tesztelési szemlélet.

4.7 A Felhasznált Szoftverek

4.8 Programozási nyelvek és technológiák

Elem	Szerep
JavaScript (ES module)	frontend és backend logika
HTML5	oldalstruktúra
CSS3	megjelenítés, reszponzív stílusok
SQL	adatbázis-séma és adatok kezelése
C#	asztali admin alkalmazás logikája
XAML	WPF felhasználói felület leírása

4.9 Könyvtárak, keretrendszerek

Elem	Szerep
React 19	frontend komponensrendszer
React Router DOM	kliensoldali útvonalkezelés
Vite	frontend build és dev szerver
Framer Motion	animációk

Backend Fejlesztés (REST API)

Elem	Szerep
Axios	API kommunikáció
Express 5	REST API
bcrypt	jelszó hash-elés
jsonwebtoken	JWT kezelés
mysql2	MySQL kapcsolat
dotenv	környezeti változók betöltése
cors	CORS kezelés
.NET 8 Desktop	WPF futtatókörnyezet

4.10 Fejlesztői környezet és eszközök

Elem	Szerep
Visual Studio Code	frontend és backend fejlesztés
Node.js v24.14.0	jelenlegi ellenőrzéskor használt futtatókörnyezet
npm workspaces	monorepo jellegű csomagkezelés

Backend Fejlesztés (REST API)

Elem	Szerep
XAMPP / MySQL	helyi adatbázis-kiszolgáló
Microsoft Word vagy kompatibilis szerkesztő	végleges dokumentáció formázása esetén
Visual Studio / .NET SDK	WPF asztali kliens buildelése

5 Frontend Fejlesztés

5.1 Webes kliens

A frontend React alapú egyoldalas alkalmazás. A belépési pont a `src/App.jsx`, ahol a teljes útvonalstruktúra és az `AuthProvider` konfiguráció található. A felhasználói élményt animációk, gördülő szekciók, karusszel és több lépéses foglalási folyamat támogatja.

5.1.1 Fő útvonalak

Útvonal	Funkció	Védelem
<code>/</code>	főoldal, előnyök, fake review-k, beágyazott terméklista és kapcsolat	publikus
<code>/termekek</code>	teljes terméklista	publikus
<code>/termekek/:id</code>	szerver részletei	publikus
<code>/termekek/:id/foglalas</code>	foglalási wizard	védett
<code>/kapcsolat</code>	kapcsolat oldal	publikus
<code>/ugyfelportal/login</code>	bejelentkezés	publikus
<code>/ugyfelportal/register</code>	regisztráció	publikus
<code>/ugyfelportal/oauth/callback</code>	OAuth visszatérési oldal	publikus
<code>/ugyfelportal/dashboard</code>	foglalások és mentorértékelések	védett

Útvonal	Funkció	Védelem
<code>/ugyfelportal/profil</code>	profiloldal	védett

5.1.2 API integráció

- központi axios kliens `src/services/api.js`;
- a token automatikusan hozzáadódik minden kéréshez;
- 401 hiba esetén automatikus kijelentkeztetés és visszairányítás történik;
- a Vite proxy `/api` kéréseket helyben a backend felé irányítja.

5.2 Mobil kliens -nincs-

A projekt jelenlegi állapotában nem tartalmaz külön Android vagy iOS mobilalkalmazást. A felhasználói kiszolgálás reszponzív webes felületen keresztül történik, ezért mobil böngészőből is használható, de külön React Native, Flutter vagy PWA csomag nem készült hozzá.

5.3 Asztali kliens

A vizsgaremekhez tartozik egy különálló WPF alapú admin kliens is. Ennek részletes ismertetése az Asztali Alkalmazás Fejlesztés fejezetben található, mert nem a webes frontend része, hanem önálló adminisztrációs modul.

5.4 Fontos frontend funkciók

- a kezdőoldal fake review-kat tölt be a backendből;
- a terméklista kereséssel, üzemeltető szerinti szűréssel és rendezéssel működik;
- a szerverrészlet oldal dinamikusan generált SVG illusztrációt használ;
- a foglalási folyamat 5 lépéses.
- a dashboard megjeleníti a foglalásokat és a mentorértékelést;
- a profiloldal base64 formájú profilképet is képes menteni;
- a navbar mobilnézettel és profilmenüvel is rendelkezik.

5.5 Fontos megjegyzések a jelenlegi implementációról

- a mentorok listája nem adatbázisból érkezik, hanem a `src/data/mentors.js` állományban statikusan szerepel;
- a mentorhoz rendelt időpontok a frontendben körkörös kiosztással jönnek létre az elérhető slotokból;
- a kapcsolatfelvételi űrlap jelenleg nem küld valódi e-mailt, csak tájékoztató alertet jelenít meg;
- a webes projekt mellett külön admin desktop kliens is létezik, amely ugyanazt a backendet használja.

6 Asztali Alkalmazás Fejlesztés

6.1 Áttekintés

A CyberNest_Admin egy különálló, .NET 8 alapú WPF asztali alkalmazást tartalmaz. Ez a modul kifejezetten admin célra készült, és ugyanahhoz a REST API-hoz kapcsolódik, mint a webes kliens.

6.2 Technológia

- C#
- WPF
- XAML
- .NET 8 (net8.0-windows)
- HttpClient alapú API-kommunikáció

6.3 Fő funkciók

Funkció	Állapot	Megjegyzés
Admin bejelentkezés	Teljesült	csak admin role esetén enged belépni
Felhasználók listázása	Teljesült	kereséssel együtt
Új felhasználó hozzáadása	Teljesült	admin/user role választható
Felhasználó törlése	Teljesült	API-n keresztül
Felhasználó módosítása	Teljesült	név, e-mail, jogosultság
Eszközök listázása	Teljesült	kereséssel együtt
Új eszköz létrehozása	Teljesült	üzemeltető kiválasztással

Funkció	Állapot	Megjegyzés
Eszköz módosítása	Teljesült	jelenlegi lista alapján
Eszköz törlése	Teljesült	API-n keresztül
Üzemeltetők listázása	Teljesült	kereséssel együtt
Üzemeltető hozzáadása	Teljesült	API-n keresztül
Üzemeltető módosítása	Teljesült	API-n keresztül
Üzemeltető törlése	Teljesült	API-n keresztül
Foglalások listázása	Teljesült	admin tokennel
Foglalások törlése	Teljesült	kijelölt rekord alapján
Értékelések kezelése	Előkészített	menüpont létezik, de nincs teljes implementáció
Naplózás kezelése	Előkészített	menüpont létezik, de nincs teljes implementáció

6.4 Felépítés

Fájl	Szerep
MainWindow.xaml	teljes admin felület UI-ja

Fájl	Szerep
<code>MainWindow.xaml.cs</code>	eseménykezelők és panelváltások
<code>ApiService.cs</code>	backend végpontok hívása
<code>Felhasznalo.cs</code>	loginválasz és user modell
<code>Eszkoz.cs</code>	eszközmodell
<code>Foglalas.cs</code>	foglalásmodell
<code>Uzemelteto.cs</code>	üzemeltetőmodell
<code>Ertekeles.cs</code>	jelenleg üres placeholder
<code>Naplo.cs</code>	jelenleg üres placeholder
<code>Stilusok/*.xaml</code>	újrafelhasználható WPF stílusok

6.5 Működés

1. Az admin a bejelentkező képernyőn e-maillal és jelszóval hitelesít.
2. A kliens a `POST /api/auth/login` végpontot hívja meg.
3. Sikeres válasz esetén eltárolja a JWT tokent és a bejelentkezett admin adatait.
4. A menüsávból külön panelek nyithatók meg a felhasználók, eszközök, üzemeltetők és foglalások kezelésére.
5. A listanézetek élő keresést támogatnak.
6. Az admin műveletek mind a közös REST API-n keresztül futnak.

6.6 Fontos technikai megfigyelések

- az alkalmazás fix méretű, 800x450 pixeles ablakban működik;
- az API alapkiszolgálója a desktop kliensben: <http://localhost:5050/api/>;
- a desktop kliens közvetlen adatbázis-hozzáférés helyett kizárólag REST API-t használ;
- az Ertekeles.cs és Naplo.cs osztályok még nincsenek érdemben kidolgozva;
- a menüpontok egy része csak előkészített placeholder;
- a bejelentkezés utáni állapotfrissítés "aktív" értéket küld, miközben a backend adatmodellje aktiv enumot használ, ez potenciális konzisztenciahiba.

7 Projektmenedzsment

7.1 Projekttervezés

A projekt célja egy iskolai vizsgaremekhez méltó, több technológiát integráló, valós használati folyamatot modellező alkalmazás létrehozása volt. A fókuszterületek:

- teljes stack megvalósítás egy repón belül;
- adatbázis-tervezés és automatikus inicializáció;
- felhasználókezelés és jogosultságkezelés;
- modern, reszponzív frontend;
- dokumentálható, bemutatható foglalási folyamat.

7.2 Célkitűzések és határidők

A repó nem tartalmaz külön projekttervező fájlt, ezért az alábbi ütemezés a meglévő kódbázis szerkezete alapján rekonstruált, dokumentációs célú idővonal.

gantt

```

title CyberNest projekt - rekonstruált ütemezés
dateFormat YYYY-MM-DD

section Tervezés
Követelményelemzés és UI koncepció      :done, 2026-01-13, 7d

section Backend
Adatbázis-terv és alap API                :done, 2026-01-20, 14d
Auth, JWT, jogosultságkezelés             :done, 2026-02-03, 10d
Foglalás és értékelési logika            :done, 2026-02-13, 12d

section Frontend
Főoldal, terméklista, részletező         :done, 2026-02-03, 19d
Foglalási wizard és dashboard             :done, 2026-02-24, 14d
Profil, OAuth, finomhangolás              :done, 2026-03-10, 14d

section Zárás
Tesztelés, hibajavítás, dokumentáció     :active, 2026-03-24, 15d

```

7.3 Csapatmunka és Verziókövetés

7.3.1 Feladatmegosztás

Terület	Résztvevők
Frontend felületek	Király Gellért, Macza Ádám
Backend API és üzleti logika	Király Gellért, Macza Ádám
Adatbázis-terv és SQL	Király Gellért, Macza Ádám
Desktop terület	Kiss Richárd Bendegúz

7.3.2 Verziókövetés

A projekt Git alapú repositoryban található. A kódszerkezet alapján a javasolt csapatmunka-folyamat:

- main vagy központi stabil ág;
- külön feature branchek új funkciókhoz;
- commitonként jól elkülöníthető fejlesztési egységek;
- merge utáni integrált tesztelés.

7.4 Kockázatelemzés

Kockázat	Valószínűség	Hatás	Enyhítés
MySQL nem elérhető a helyi gépen	Közepes	Magas	XAMPP/MySQL ellenőrzése, <code>.env</code> helyes kitöltése, 3306/3307 fallback

Projektmenedzsment

Kockázat	Valószínűség	Hatás	Enyhítés
OAuth környezeti változók hiánya	Magas	Közepes	A rendszer hibaüzenettel jelzi a hiányzó kulcsokat
Időpont ütközés	Közepes	Magas	Foglalás előtt státusz ellenőrzés és azonnali <code>reserve()</code> hívás
Statikus mentoradatok	Magas	Közepes	Mentorok adatbázisba költöztetése a következő verzióban
Kapcsolat űrlap nem küld levelet	Magas	Alacsony	SMTP / email API integráció
Lint hibák a frontendben	Magas	Közepes	Refaktorálás <code>AuthContext</code> és <code>Contact</code> komponensekben
Automatizált tesztek hiánya	Magas	Magas	Jest / Vitest / Supertest bevezetése

8 Fejlesztői Dokumentáció

8.1 Részletes feladatspecifikáció

A CyberNest fejlesztői oldalról egy többkomponensű rendszer, amelynek fő célja a szerverkonfigurációk böngészése, mentorált foglalások kezelése és adminisztratív karbantartása. A megvalósítás során az alábbi fő funkciókat kellett biztosítani:

- nyilvános terméklista és részletező nézet;
- hitelesítés lokális és OAuth belépéssel;
- több lépéses foglalási folyamat;
- időpontkezelés és státuszváltás;
- profilkezelés és mentorértékelés;
- admin és desktop oldali CRUD műveletek.

8.2 Entitások leírása

8.2.1 Eszköz entitás

Az `eszkoz` entitás reprezentálja a foglalható szervercsomagokat. Tulajdonságai között szerepel a leírás, CPU, RAM, HDD és a kapcsolódó üzemeltető azonosítója.

8.2.2 Felhasználó entitás

A `felhasznalo` entitás a regisztrált és adminisztratív szerepkörű rendszerhasználókat írja le. Tárolja a név, e-mail, jelszóhash, szerepkör, állapot és profilkép adatait.

8.2.3 Foglalás entitás

A `foglalas` entitás kapcsolja össze a felhasználót, a választott szerverkonfigurációt, az időpontot és a mentoradatokat. Ebben a táblában található a foglalási folyamat során megadott ügyfél- és kapcsolattartási adatok is.

8.2.4 Kapcsolat az entitások között

Az alkalmazás legfontosabb kapcsolatai az `eszkoz`, `felhasznalo`, `idopont`, `foglalas`, `ertekeles`, `mentor_ertekeles` és `log` entitások között jönnek létre. Ezek a kapcsolatok biztosítják, hogy egy foglaláshoz időpont, felhasználó, értékelés és naplóbejegyzések is hozzárendelhetők legyenek.

8.3 Használati esetek (use-case)

8.3.1 Vendég

- böngészi a főoldalt és a terméklistát;
- megnyitja az egyes szervercsomagok részleteit;
- regisztrál vagy bejelentkezik.

8.3.2 Regisztrált felhasználó

- foglalást indít és időpontot választ;
- megtekinti, törli és értékeli a saját foglalásait;
- módosítja a profiladatait.

8.3.3 Admin

- kezeli a felhasználókat, eszközöket, időpontokat és üzemeltetőket;
- lekérdezi az értékeléseket és logokat;
- desktop kliensből is adminisztrálhatja az adatokat.

8.4 Nézetek

Az alkalmazás fő nézetei a főoldal, a terméklista, az egyedi szerverrészletező oldal, a többlépéses foglalási wizard, a dashboard, a profiloldal, a kapcsolat oldal, valamint a külön WPF admin képernyők. Ezek a nézetek együtt fedik le a vendég, felhasználó és admin szerepkörökhöz tartozó használati eseteket.

8.5 API végpontok meghatározása

A REST API végpontokat a frontend nézetek adatfeldolgozási igényei alapján határoztuk meg. Külön erőforráscsoport tartozik az autentikációhoz, az eszközökhöz, az időpontokhoz, a foglalásokhoz, a felhasználókhoz, az üzemeltetőkhez, az értékelésekhez és a naplózáshoz. A publikus és védett műveletek szétválasztását JWT token és admin middleware biztosítja.

8.6 Projektstruktúra

```
vizsgaremek/  
├─ .gitignore  
├─ package-lock.json
```


Fejlesztői Dokumentáció

```
├─ CyberNest_teljes_dokumentacio.md
├─ CyberNest_Admin/
│   └─ CyberNest_Admin.slnx
│   └─ CyberNest_Admin/
│       └─ CyberNest_Admin.csproj
│       └─ MainWindow.xaml
│       └─ MainWindow.xaml.cs
│       └─ ApiService.cs
│       └─ Felhasznalo.cs
│       └─ Eszkoz.cs
│       └─ Foglalas.cs
│       └─ Uzemelteto.cs
│       └─ Stilusok/
└─ website/
    └─ cybernest.bat
    └─ package.json
    └─ package-lock.json
    └─ backend/
        └─ .env
        └─ package.json
        └─ package-lock.json
        └─ server.js
        └─ setup.sql
        └─ test3backup.sql
        └─ config/
            └─ database.js
        └─ controllers/
            └─ authController.js
            └─ ertekelesController.js
            └─ eszkozController.js
            └─ felhasznaloController.js
            └─ foglalasController.js
            └─ idopontController.js
            └─ logController.js
            └─ mentorErtekelesController.js
            └─ uzemeltetoController.js
        └─ middleware/
            └─ auth.js
```

```
|   └─ models/
|     └─ Ertekeles.js
|     └─ Eszkoz.js
|     └─ Felhasznalo.js
|     └─ Foglalas.js
|     └─ Idopont.js
|     └─ Log.js
|     └─ MentorErtekeles.js
|     └─ Uzemelteto.js
|   └─ routes/
|     └─ auth.js
|     └─ ertekeles.js
|     └─ eszkoz.js
|     └─ felhasznalo.js
|     └─ foglalas.js
|     └─ idopont.js
|     └─ log.js
|     └─ uzemelteto.js
|   └─ services/
|     └─ api.js
|   └─ utils/
|     └─ validation.js
└─ frontend/
  └─ .gitignore
  └─ eslint.config.js
  └─ index.html
  └─ package.json
  └─ vite.config.js
  └─ public/
    └─ favicon.png
    └─ review-avatars/
  └─ src/
    └─ App.jsx
    └─ main.jsx
    └─ assets/
      └─ icons/
    └─ context/
      └─ AuthContext.jsx
```

```
├─ data/
|   └─ mentors.js
├─ layout/
|   ├── Footer.jsx
|   ├── Navbar.jsx
|   └─ ProtectedRoute.jsx
├─ pages/
|   ├── BookingFlow.jsx
|   ├── Contact.jsx
|   ├── Dashboard.jsx
|   ├── Home.jsx
|   ├── Login.jsx
|   ├── OAuthCallback.jsx
|   ├── Products.jsx
|   ├── Profile.jsx
|   ├── Register.jsx
|   └─ ServerDetails.jsx
├─ services/
|   └─ api.js
├─ styles/
|   ├── Auth.css
|   ├── BookingFlow.css
|   ├── Contact.css
|   ├── Dashboard.css
|   ├── Footer.css
|   ├── Home.css
|   ├── Navbar.css
|   ├── Products.css
|   ├── Profile.css
|   ├── ServerDetails.css
|   └─ global.css
└─ utils/
    └─ validation.js
```

8.7 Környezeti változók

Változó	Szerep
DB_HOST	adatbázis hoszt
DB_USER	adatbázis felhasználó
DB_PASSWORD	adatbázis jelszó
DB_NAME	adatbázis neve
JWT_SECRET	JWT aláíró kulcs
PORT	backend port
CLIENT_URL	engedélyezett kliens origin
GOOGLE_CLIENT_ID	Google OAuth kliensazonosító
GOOGLE_CLIENT_SECRET	Google OAuth titok
GOOGLE_REDIRECT_URI	Google callback URL
GITHUB_CLIENT_ID	GitHub OAuth kliensazonosító
GITHUB_CLIENT_SECRET	GitHub OAuth titok
GITHUB_REDIRECT_URI	GitHub callback URL

Változó	Szerep
FACEBOOK_APP_ID	jelenleg nincs használva
FACEBOOK_APP_SECRET	jelenleg nincs használva
FACEBOOK_REDIRECT_URI	jelenleg nincs használva

8.8 Részletes feladatspecifikáció, algoritmusok

8.8.1 1. Regisztráció

1. A kliens elküldi a nevet, emailt és jelszót.
2. A backend normalizálja a szöveges mezőket.
3. Regex alapú validáció fut a névre, emailre és jelszóra.
4. A rendszer ellenőrzi, létezik-e már ilyen email.
5. A jelszó bcrypt-tel hash-elődik.
6. Az új felhasználó bekerül a `felhasznalo` táblába.

8.8.2 2. Bejelentkezés

1. A felhasználó elküldi email + jelszó adatait.
2. A rendszer megkeresi az emailhez tartozó rekordot.
3. bcrypt összehasonlítás történik.
4. Siker esetén JWT token készül 24 órás lejáratral.
5. A frontend localStorage-ba menti a tokenet és a user objektumot.

8.8.3 3. Foglálás létrehozása

1. A kliens kiválaszt egy eszközt, mentort és időpontot.
2. A backend ellenőrzi az eszköz- és időpontazonosítót.
3. A kapcsolattartási adatok regex validáción esnek át.
4. A backend megvizsgálja, hogy az időpont még `available` állapotú-e.
5. Ha igen, az időpont státusza `reserved` lesz.
6. A rendszer ellenőrzi, hogy a dátum nem múltbeli és a kezdőnap <= zárónap.

7. Létrejön a `foglalas` rekord.
8. Hiba esetén a rendszer felszabadítja az időpontot.

8.8.4 4. Mentorértékelés mentése

1. A dashboard elküldi a foglалás azonosítóját, pontszámot és review-t.
2. A backend ellenőrzi, hogy a foglалás a felhasználóhoz tartozik-e.
3. A pontszám 1 és 5 közé kell eszen, 0.5 lépésközzel.
4. A review legfeljebb 1200 karakter lehet.
5. A rendszer UPSERT művelettel létrehozza vagy frissíti az értékelést.
6. Ezután újraszámolja a mentor átlagpontszámát.

8.9 Az alkalmazott fejlesztői eszközök

Eszköz	Szerep
Visual Studio Code	webes frontend és backend fejlesztése
Visual Studio / .NET SDK	WPF admin kliens fordítása és futtatása
Node.js + npm	frontend build, csomagkezelés és fejlesztői futtatás
MySQL / XAMPP	helyi adatbázis szolgáltatás
Git	verziókövetés és csapatmunka
Microsoft Word	végleges dokumentáció szerkesztése

8.10 Fejlesztői megjegyzések

- a backend induláskor automatikusan létrehozza az adatbázist és futtatja a `setup.sql` szkriptet;
- a rendszer egy alap admin felhasználót is biztosít fejlesztői környezethez (`admin@local.com` / `admin123`);

- a `frontend/vite.config.js` a proxyt `http://localhost:5050` címre irányítja, ezért a backend PORT változót célszerű ehhez igazítani;
- az `AuthContext.jsx` jelenlegi formában lint hibát ad, mert egyszerre exportál context helper függvényt és komponenst, illetve effecten belül közvetlenül állít state-et.

9 Tesztelési Dokumentáció

9.1 Általános tesztelési stratégia

A repository jelenlegi állapotában nincs külön automatikus unit- vagy integrációs tesztcsomag. A tesztelés ezért három részből áll:

- statikus ellenőrzés (lint);
- build ellenőrzés;
- kézi funkcionális tesztesetek megtervezése és végrehajtása.

9.2 Tesztelési táblázat

ID	Teszt neve	Bemenet / lépés	Elvárt eredmény
TC-01	Érvényes regisztráció	új név, új email, szabályos jelszó	sikeres regisztráció, átirányítás login oldalra
TC-02	Hibás email regisztrációnál	peldaemail.hu	hibaüzenet: érvényes email szükséges
TC-03	Hibás jelszó login során	létező email + rossz jelszó	401 hiba, sikertelen belépés
TC-04	Védett foglalás vendégként	közvetlenül megnyitott /termek/:id/foglalas	átirányítás login oldalra
TC-05	Üres kötelező foglalási mezők	név / email / telefon nélkül továbblépés	kliensoldali hibaüzenet
TC-06	Múltbeli dátum foglalás	múltbeli berlesi_kezdet	backend elutasítja a foglalást

ID	Teszt neve	Bemenet / lépés	Elvárt eredmény
TC-07	Mentorértékelés 6 ponttal	pontszám = 6	backend validációs hiba
TC-08	Profilkép túl nagy fájl	20 MB-nál nagyobb kép feltöltése	hibaüzenet, mentés nem történik meg

9.3 Tesztesetek

9.3.1 1. Normál használat - érvényes regisztráció

- Tesztelt funkció: felhasználói regisztráció
- Bemenet: új név, egyedi e-mail cím, szabályos jelszó
- Elvárt eredmény: a rendszer létrehozza a felhasználót és átirányít a bejelentkezési oldalra
- Kapott eredmény: a funkció a dokumentált folyamat szerint működik
- Teszt eredménye: sikeres

9.3.2 2. Negatív teszt - hibás e-mail cím

- Tesztelt funkció: regisztrációs validáció
- Bemenet: peldaemail.hu
- Elvárt eredmény: a felület hibaüzenetet jelenít meg, mentés nem történik
- Kapott eredmény: a hibás formátum elutasításra kerül
- Teszt eredménye: sikeres

9.3.3 3. Bolondbiztos teszt - túl nagy profilkép

- Tesztelt funkció: profilkép feltöltése
- Bemenet: 20 MB-nál nagyobb fájl
- Elvárt eredmény: a rendszer nem menti el a képet, és hibaüzenetet küld
- Kapott eredmény: a kliens validáció leállítja a műveletet
- Teszt eredménye: sikeres

9.4 Automatizált tesztelés

A repository jelenlegi állapotában nincs külön Jest, Vitest, Supertest vagy más automatikus tesztsomag. Ennek ellenére futtathatók olyan technikai ellenőrzések, amelyek visszajelzést adnak a fordíthatóságról és a kódbázis állapotáról.

9.5 Technikai ellenőrzések (2026. április 7.)

Ellenőrzés	Parancs	Eredmény
VT-01	<code>node -v</code>	sikeres, Node.js v24.14.0
VT-02	<code>npm run build -w frontend</code>	sikeres production build
VT-03	<code>npm run lint -w frontend</code>	sikertelen, 3 frontend hiba
VT-04	<code>dotnet build CyberNest_Admin.csproj</code>	sikeres build, 0 figyelmeztetés, 0 hiba

9.6 Talált hibák

1. frontend/src/context/AuthContext.jsx

A fájl egyszerre exportál komponenst és segédfüggvényt, emiatt sérül a `react-refresh/only-export-components` szabály.

1. frontend/src/context/AuthContext.jsx

Az effektben közvetlen `setState` hívások futnak, ami lint figyelmeztetés szerint fölösleges újrenderelést okozhat.

2. frontend/src/pages/Contact.jsx

A `motion` import szerepel, de közvetlenül nincs felhasználva a fájlban.

3. Funkcionális hiányosság

A kapcsolatfelvételi űrlap nem küld valódi üzenetet, csak lokális értesítést ad.

4. Architektúrális hiányosság

A mentorok és a mentor-slot hozzárendelés jelenleg nem backend oldali adatmodellből származik.

5. Desktop kliens konzisztenciahiba

A WPF kliens bejelentkezéskor "aktív" állapotot küld a felhasználófrissítéshez, miközben a backend az aktiv enumértéket használja.

6. Desktop kliens hiányzó moduljai

Az értékelés és naplózás menüpontok a felületen elérhetők, de a tényleges adatkezelő implementációjuk még nem készült el.

9.7 Tesztelési környezet

- operációs rendszer: Windows alapú fejlesztői környezet;
- frontend futtatókörnyezet: Node.js v24.14.0;
- frontend build eszköz: Vite;
- desktop build környezet: .NET 8 SDK;
- adatbázis: MySQL / XAMPP;
- ellenőrzéshez használt kliens: modern Chromium-alapú böngésző.

10 Felhasználói Dokumentáció

10.1 A program általános specifikációja

A CyberNest egy webes szolgáltatás, amelyen a felhasználó kiválaszthat egy szervercsomagot, megnézheti annak részleteit, majd mentor és időpont választásával foglalást adhat le. A bejelentkezett felhasználó később a saját felületén visszanezheti, törölheti és értékelheti a foglalásait, illetve módosíthatja a profilját.

10.2 Rendszerkövetelmények

10.2.1 Hardverkövetelmények

Minimum helyi bemutató / fejlesztői környezethez

- kétmagos processzor;
- 4 GB RAM;
- legalább 2 GB szabad tárhely;
- stabil hálózati kapcsolat a csomagtelepítéshez.

Ajánlott

- négymagos processzor;
- 8 GB RAM;
- SSD háttértár;
- 1920x1080 képernyőfelbontás.

10.2.2 Szoftverkövetelmények

Helyi futtatáshoz

- Windows 10 vagy Windows 11;
- Node.js és npm;
- MySQL vagy XAMPP MySQL szolgáltatás;
- modern böngésző: Chrome, Edge vagy Firefox.

Desktop admin klienshez

- Windows 10 vagy Windows 11;
- .NET 8 Desktop Runtime;
- futó CyberNest backend szolgáltatás;

- admin jogosultságú felhasználó.

Használathoz éles környezetben

- internetkapcsolat;
- modern böngésző JavaScript támogatással.

10.3 A teljes program telepítése

10.3.1 A weboldal és backend - 1. Módszer: batch fájlal közvetlen repóból (Ajánlott*)

1. A projektet töltsük le vagy klónozzuk a gépre.
2. Ha zip-et töltött le a böngésző akkor csomagoljuk ki a mappa tartalmát.
3. Ellenőrizzük, hogy a XAMPP a `C:\xampp` könyvtárba van-e telepítve.
4. Ellenőrizzük, hogy a `website/backend/.env` fájl ki van-e töltve.
5. Navigáljunk a `website` mappába majd válasszuk ki a `cybernest.bat` állományt.
6. Futtassuk a `CyberNest.bat` állományt.
7. A script elindítja az Apache és MySQL szolgáltatást, telepíti a csomagokat és megnyitja a böngészőt.

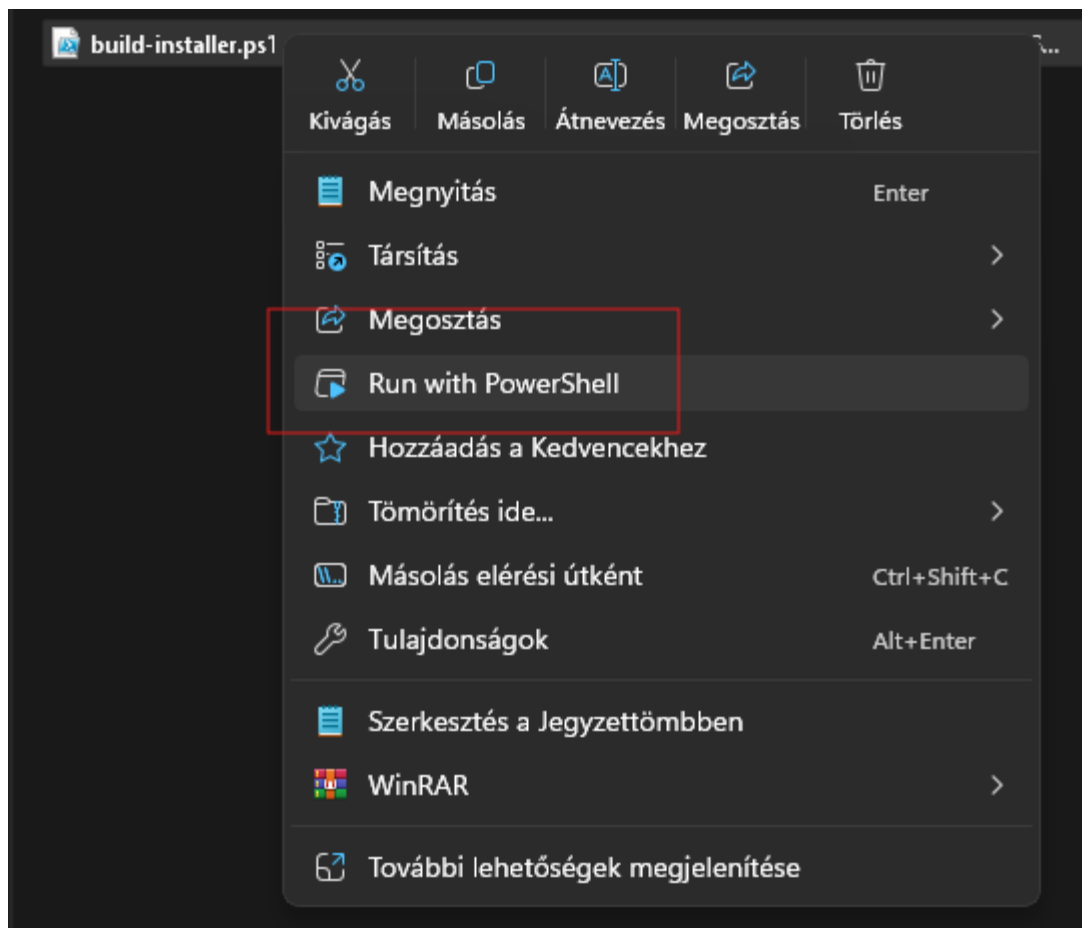
10.3.2 A weboldal és backend - 2. Módszer: kézi indítás (Nem ajánlott)

1. Nyissunk terminált a `website` mappában.
2. Futtassuk: `npm install`
3. Indítsuk el a MySQL szolgáltatást.
4. Ellenőrizzük a `.env` fájlt, különösen a `DB_*`, `JWT_SECRET`, `PORT`, `CLIENT_URL` mezőket.
5. Futtassuk: `npm run dev`
6. Nyissuk meg a böngészőben a `http://localhost:5173` címet.

10.3.3 Az asztali alkalmazás telepítése közvetlen a repóból letöltve (Nem ajánlott)

1. A projekt letöltése vagy klónozása után navigáljunk a következő mappába:
2. `Desktop/CyberNest_Admin/scripts`

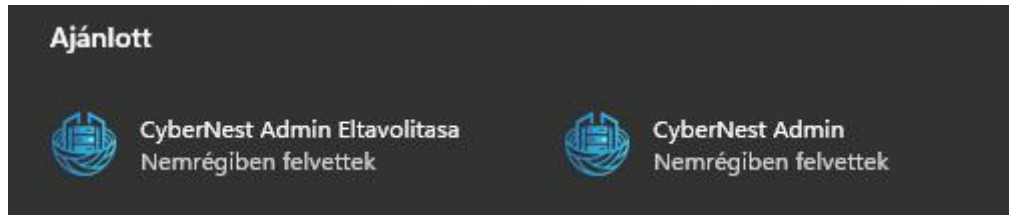
3. Jobb kattintsunk az egéren a következő file-ra `build-installer.ps1` és válasszuk a futtatás PowerShellben opciót.



ábra 6 Telepítő segédlet, hogy mit válasszunk konkrétan.

4. A telepítés során a program Létrehozza a telepítő exénket, aminek segítségével tudjuk a számítógépünkre telepíteni az alkalmazást.
5. Alapértelmezetten ez a `Desktop/CyberNest_Admin/artifacts/installer/` helyre kerül itt kattintsunk duplán a `CyberNest_Admin_Setup.exe`-re és települni fog a program a számítógépre ezt egy sikeres üzenet erősíti meg és elindul a program.
6. a Start menüben is megjelenik az alkalmazás, valamint egy uninstaller (eltávolító) is telepítésre kerül.

7. A telepítés után az alkalmazás az asztali ikonról vagy a Start menüből indítható.



ábra 7 Startmenü ikonok

10.3.4 A Teljes projekt Egyszerű szakszerű telepítése (Feltétlenül ajánlott!!)

1. Töltsük le a Github Release kiadást. Ez tartalmazni fogja a szükséges telepítőket és fileokat, dokumentációkat, lényegében mindent.
2. Ha zip-ben töltötte le a böngészőnk akkor csomagoljuk ki a mappa tartalmát!
3. A mappában két fő rész található:
 - az asztali admin alkalmazás telepítője
 - a weboldal es backend rendszer indító fájlja es kapcsolódó mappája
4. Indítás menete:
5. Kattints duplán a **CyberNest.exe** fájlra.
6. Az indító automatikusan elindítja a website mappában található szükséges folyamatokat.
7. A rendszer elindítja a backend es frontend részt.
8. Sikeres indítás után a weboldal böngészőben nyílik meg.

Fontos:

- A **CyberNest.exe** és a website mappa mindig maradjon egy helyen.
 - A website mappa tartalmat nem szabad külön áthelyezni.
 - A webes rendszer futtatásához a projekt beállításaitól függően szükséges lehet a Node.js, valamint a XAMPP/MySQL megfelelő elérhetősége.
 - Ha az indítás nem sikerül, ellenőrizni kell, hogy minden szükséges komponens telepítve van-e.
9. Majd a másik **CyberNest_Admin_Setup.exe** -t is indítsuk el ezzel fog a projekt Admin alkalmazása települni.

Fontos:

- Az admin alkalmazás futtatásához szükséges, hogy fusson a korábban említett `CyberNest.exe` mert ez tartalmazza a szükséges adatbázist az alkalmazás eléréséhez.

10.4 Fontos telepítési megjegyzések

- a backend induláskor automatikusan létrehozza a hiányzó adatbázist és táblákat;
- a rendszer demo adatokat is betölt;
- fejlesztői környezetben az alap admin belépés: `admin@local.com` / `admin123`;
- ha a frontend nem éri el a backendet, ellenőrizni kell, hogy a Vite proxy célpontja és a backend port megegyezik-e.
- a desktop kliens alapértelmezésben a `http://localhost:5050/api/` végpontot használja, ezért a backendet ezen a címen kell futtatni, vagy a kliensben kell átírni az alapkiszolgálót.

10.5 A program használatának részletes leírása

10.5.1 1. Főoldal használata

- A nyitóképernyő röviden bemutatja a szolgáltatást.
- A Felfedezés gomb a termékek szekcióhoz görget.
- A kezdőlapon elérhetők a kiemelt előnyök és az ügyfélvélemények is.
- Van egy szakasz, ahol a mentornak jelentkezést is népszerűsítjük az oldalon.
- Cégeknek is kínálunk különleges ajánlatokat.

10.5.2 2. Termékek böngészése

- Nyisd meg a `Termékek` menüpontot.
- A keresőmezőben CPU, RAM, HDD, leírás vagy üzemeltető alapján lehet keresni.
- Az `Üzemeltető` legördülővel szűkíthető a lista.
- A `Rendezés` mezővel ID vagy RAM alapján rendezhető a találati lista.

- A **Kiválasztás** gombra kattintva megnyílik a szerver részletes oldala.

10.5.3 3. Regisztráció és bejelentkezés

- A **Bejelentkezés** menüpont alatt lokális vagy közösségi belépés választható.
- Új felhasználó a **Regisztráció** oldalon hozhat létre fiókot.
- A rendszer ellenőrzi a felhasználónév, az email és a jelszó formátumát.

10.5.4 4. Foglalási folyamat

1. Nyisd meg a kívánt szerver részletes oldalát.
2. Kattints a Foglалás megkezdése gombra.
3. Válassz mentort.
4. Válassz elérhető időpontot.
5. Töltsd ki a foglalási adatokat.
6. Az összegzés oldalon ellenőrizd az adatokat.
7. Fogadd el az ÁSZF-et.
8. Kattints a Foglалás véglegesítése gombra.

10.5.5 5. Dashboard használata

- A **Foglalásaim** blokkban látható minden saját foglalás.
- A felhasználó törölheti a saját foglalását.
- A **Mentor értékelések** blokkban fél csillagos lépésközzel pontszám adható.
- Mentés után a rendszer frissíti a mentor átlagértékét.

10.5.6 6. Profil használata

- A profiloldalon módosítható a felhasználónév és az email cím.
- Új jelszó is beállítható, megerősítéssel együtt.

10.5.7 7. Desktop admin kliens használata

1. Indítsd el a webes backend szolgáltatást.
2. Nyisd meg a **CyberNest_Admin** alkalmazást.
3. Add meg az admin e-mail címet és jelszót.
4. Sikeres belépés után a felső menüsávból kiválasztható a kezelendő modul.

5. A Felhasználók, Eszközök, Üzemeltetők és Foglалás menük alatt listázás, hozzáadás, módosítás és törlés végezhető.
6. Az Értékelések és Naplózás menüpontok jelen vannak, de a mostani verzióban nem teljesen működnek.
7. Az alkalmazás Napszakhoz megfelelően üdvözi a felhasználót.

10.6 FAQ

- **Kérdés:** Miért nem tudok foglalni bejelentkezés nélkül? **Válasz:** A foglalás védett funkció, ezért hitelesítés szükséges.
- **Kérdés:** Mi történik, ha egy időpontot közben más foglal le? **Válasz:** A backend ellenőrzi az időpont státuszát, és elutasítja a már foglalt slotokat.
- **Kérdés:** Miért nem megy a Google vagy GitHub belépés? **Válasz:** Valószínűleg hiányzik valamelyik OAuth környezeti változó a backend .env fájlból.
- **Kérdés:** Hol látom a saját foglalásaimat? **Válasz:** A bejelentkezés után a Foglалásaim oldalon.
- **Kérdés:** A kapcsolat űrlap miért nem küld valódi üzenetet? **Válasz:** A jelenlegi verzióban ez még statikus felület, backend email-küldés nélkül.
- **Kérdés:** Mire szolgál a desktop alkalmazás? **Válasz:** A desktop kliens egy adminisztrációs felület, amellyel a felhasználók, eszközök, üzemeltetők és foglalások kezelhetők.

11 Összefoglalás

A CyberNest vizsgaremek jól szemlélteti egy teljes stackes webalkalmazás megtervezését és megvalósítását. A projekt tartalmaz adatbázis-tervezést, REST API fejlesztést, jogosultságkezelést, reszponzív frontend felületet, több lépéses üzleti folyamatot és felhasználói profilkezelést is.

A rendszer külön erőssége, hogy a backend induláskor automatizált adatbázis-inicializációt végez, támogatja az OAuth-alapú belépést, és a foglалásokat összeköti a mentorértékeléssel. A projekt ugyanakkor még fejleszthető tesztelhetőség, adatmodellezés és adminisztrációs felületek terén.

11.1 Önértékelés

A projekt során sikerült elmélyíteni a tudást az alábbi területeken:

- frontend és backend együttműködés;
- SQL-alapú adatmodellezés;
- autentikációs és jogosultsági modellek;
- validáció és hibakezelés;
- több lépéses űrlapfolyamatok tervezése;
- reszponzív és animált felületkialakítás.

A legnagyobb kihívást a foglalási logika és a hitelesítés összehangolása jelentette. Emellett külön figyelmet igényelt a profilkezelés és az OAuth beléptetés integrációja is. A dokumentáció készítése során egyértelművé vált, hogy a projekt műszakilag erős alapokon áll, de további finomhangolás szükséges a hosszú távú fenntarthatósághoz.

11.2 Továbbfejlesztési Lehetőségek

- a mentorok áthelyezése statikus frontend fájlból adatbázisba;
- valódi admin felület készítése a jelenlegi admin API-k fölé;
- kapcsolatfelvételi űrlap backend email-küldéssel;
- automatizált egység- és integrációs tesztek bevezetése;
- Swagger vagy OpenAPI dokumentáció generálása;
- auditnapló bővítése részletes eseményszintekkel;
- desktop kliens forráskódjának elkészítése és a repóba integrálása;

Összefoglalás

- értesítések e-mailben vagy push csatornán;
- jogosultsági szintek bővítése külön mentor/admin/staff szerepkörökkel;
- a foglalási és értékelési folyamat teljes backend oldali konzisztencia-ellenőrzése.

12 Felhasznált Irodalom

12.1 Nyomtatott könyv

A dokumentáció elkészítése során külön nyomtatott szakkönyvre nem támaszkodtunk, a szakmai háttér elsősorban a tanult tananyagból, a projekt saját forráskódjából és a hivatalos online dokumentációkból származott.

12.2 Elektronikus dokumentumok és projektforrások

- `website/backend/server.js`
- `website/backend/config/database.js`
- `website/backend/setup.sql`
- `website/backend/routes/*`
- `website/backend/controllers/*`
- `website/backend/models/*`
- `website/frontend/src/App.jsx`
- `website/frontend/src/pages/*`
- `website/frontend/src/context/AuthContext.jsx`
- `website/frontend/src/services/api.js`
- `website/cybernest.bat`

12.3 Internetes források (weboldal, dokumentációk)

- React Documentation: <https://react.dev/>

Letöltés dátuma: 2026. április 7.

- Vite Documentation: <https://vite.dev/>

Letöltés dátuma: 2026. április 7.

- Express Documentation: <https://expressjs.com/>

Letöltés dátuma: 2026. április 7.

- MySQL 8 Reference Manual: <https://dev.mysql.com/doc/>

Letöltés dátuma: 2026. április 7.

- JSON Web Tokens Introduction: <https://jwt.io/introduction>

Felhasznált Irodalom

Letöltés dátuma: 2026. április 7.

Az főoldalon a képek a <https://www.imgix.com/> oldal képeit használják ez ellenőrizhető a képekre jobb egérgomb kattintása majd kép megnyitása új lapon opció kiválasztásával ezzel az url-t kiírja.

A mentoroknál a képek a <https://xsgames.co/randomusers/> API-t hívják és használják.

Az ER-Diagramm a <https://www.eraser.io/ai/erd-generator> oldal segítségével készült.

13 Ábrajegyzék

ÁBRA 1 FŐOLDAL EGYSZERŰ DRÓTVÁZA (WIREFRAME)	7
ÁBRA 2 TERMÉKLISTA OLDAL EGYSZERŰ DRÓTVÁZA (WIREFRAME).....	8
ÁBRA 3 FOGLALÁSI FOLYAMAT OLDAL EGYSZERŰ DRÓTVÁZA (WIREFRAME)	8
ÁBRA 4 ÜGYFÉLPORTÁL OLDAL EGYSZERŰ DRÓTVÁZA (WIREFRAME)	9
ÁBRA 5 AZ ADATBÁZIS ER DIAGRAMMJA.	12
ÁBRA 6 TELEPÍTŐ SEGÉDLET HOGYMIT VÁLASSZUNK KONKRÉTAN.	48
ÁBRA 7 STARTMENÜ IKONOK	49